Lesson 01 — How the Internet Works: Clients, Servers, and Data Transfer

> *Phase 1 · Internet Fundamentals · Date: 2026-07-19*

---

1. What is the Internet?

The Internet is millions of computers connected by **physical cables** (fiber optic under oceans, copper into buildings) and **radio waves** (WiFi, 4G/5G), all agreeing to speak the same rules called **protocols**.

Two core ideas:

1. **Physical connections** — wires and radios carrying electrical/light signals.
2. **Protocols** — shared rules so any computer can talk to any other, regardless of brand, OS, or country.

The Internet is a **network of networks** (*inter-net*): home network + office network + Google's network + ISP networks, all interconnected. Nobody owns it. There is no central computer running it.

2. Why was it created?

1960s, ARPANET (US research project). Problems it solved:

- Expensive computers at different universities could not talk to each other.
- They wanted a network with **no single point of failure** — if one path dies, traffic routes around it. That decentralization principle still defines the Internet today.

3. Client and Server (the most important concept)

Every Internet interaction has two roles:

Role	What it does	Examples
Client	*Asks* — always starts the conversation	Browser, mobile app, `curl`, Java `RestTemplate`
Server	*Answers* — sits waiting, listening for requests	NGINX, Spring Boot app, PostgreSQL

⭐ **Key insight:** client and server are **roles, not machines.**

- A server is not special hardware — it is any computer running a program that *listens*. My laptop becomes a server the moment I run `mvn spring-boot:run`.
- The same program can be both: my Spring Boot app is a **server** to browsers, but a **client** when it calls PostgreSQL.
- **Servers never initiate; clients always start the conversation.**

Analogy: the restaurant 🍔

- You (client) order: "One burger" → the **request**
- Kitchen (server) is always open, waiting, cooks → **processing**

- Waiter brings food → the **response**
- The kitchen never comes to your house asking if you're hungry (servers don't initiate).
- The kitchen serves many tables at once (one server, thousands of clients).

4. How data travels: Packets

Data is **never sent as one big chunk**. It is chopped into **packets** (~1,500 bytes each). Every packet carries:

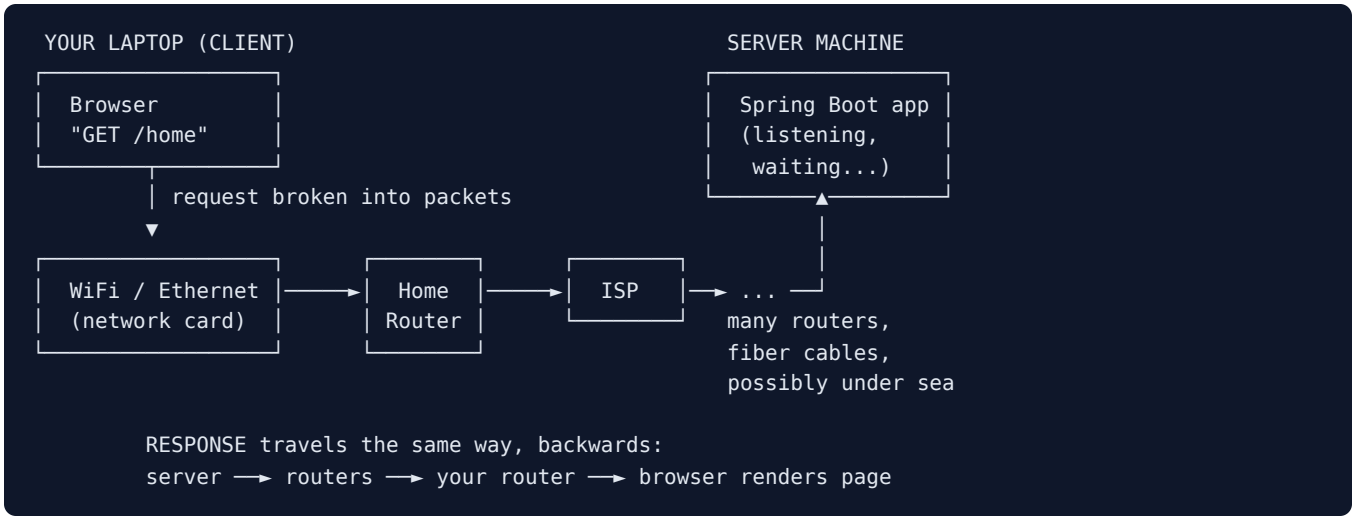
- **Destination address** — where it's going
- **Source address** — where it came from (so the reply knows the way back)
- **Sequence number** — so pieces can be reassembled in order
- A chunk of the actual data

Packets travel independently — they may even take **different physical routes** — and are reassembled at the destination.

Analogy: mailing a book 📬

Mailing a 500-page book when the post office only accepts thin envelopes: send 500 envelopes, each labeled "page 137 of 500". Some go by truck, some by plane. Receiver reorders and rebuilds the book. If envelope 137 is lost, only that one is resent — not the whole book. (The resend mechanism = TCP, covered later.)

5. Architecture diagram — the journey



Router = device whose only job is reading a packet's destination and forwarding it one step closer (like a postal sorting center). No router knows the full path — each only knows the next best hop. A typical request passes through **10–25 routers**.

6. Real-world production example

Opening `facebook.com` from Dhaka:

- Packets go through my ISP → submarine fiber cables (Bangladesh: SEA-ME-WE cables) → a Facebook data center, possibly Singapore.
- Facebook's servers are just Linux machines running server programs — same concept as my Spring Boot app, × millions.
- Response returns in ~100–300 ms. Light in fiber ≈ ⅔ speed of light → **distance = delay** → why CDNs exist (Phase 8).

7. Common beginner mistakes ❌

- Thinking "the cloud" is not physical. **AWS = someone else's computers, rented.**
- Thinking a server is special hardware. It's a *program that listens*.
- Thinking data goes directly A → B. It hops through 10–25 routers.
- Thinking WiFi is the Internet. WiFi is only the last few meters (device ↔ router). Beyond that: cables.
- Thinking a request is one piece. It's always packets.

8. Best practices (mindset) ✅

- Debugging "API is down"? Always ask: **where in the journey did it fail?** Client? Network? Server? The diagram above is a career-long debugging map.
- Assume the network **will** fail, slow down, drop packets. Production design = timeouts, retries (later phases).
- Latency is physics: **distance = delay.** Keep servers close to users.

9. Interview questions 🎤

1. What happens conceptually when a browser requests a page from a server? (client → routers → server → response)
2. What is a packet, and why split data into packets instead of sending it whole? *(fair sharing of the wire, retransmit only lost pieces, route around failures)*
3. Is "client" a property of a machine or a conversation? Give an example of one program being both.
4. What does a router do? Does it know the full path?
5. Why is a US server slower than a Singapore server for a user in Bangladesh, even if both servers are equally fast?

10. Lab commands 🧪

```
# 1. Measure round-trip time to a server
ping -c 4 google.com          # look at time=XXms

# 2. Compare distance = latency
ping -c 4 facebook.com
ping -c 4 gov.bd

# 3. See the actual router hops
traceroute google.com         # each line = one router; line 1 = home router
# install if missing: sudo apt install traceroute

# 4. Be a client without a browser
curl -v https://example.com   # ">" lines = request, "<" lines = response

# 5. Be a server (terminal 1) and its client (terminal 2)
python3 -m http.server 8000   # terminal 1 — server listening
curl http://localhost:8000    # terminal 2 — client; watch terminal 1 log it
```

11. Homework 📝

1. `traceroute` to 3 sites — local (BD), regional (India/Singapore), US (`mit.edu`). Record hop counts + ping times; explain differences in 2–3 sentences.
2. Explain to an imaginary junior (5–6 sentences, from memory): what happens when opening a website — use *client, server, packet, router*.

3. Answer the 5 interview questions from memory.
4. Thought question: a Spring Boot app calls PostgreSQL and an external payment API. List every client role and every server role.

Next lesson: Lesson 02 — IP Addresses (IPv4, IPv6, public vs private, localhost): *how does a packet know the "address" of a machine?*